Engineering
Web Development
Course Material

UNIT — ASP.NET 2.0

Themes & Master Pages

Displaying Data with the GridView Control

- Creating a Consistent Website
- ASP.NET Themes & Skins
- Master Pages Layout
- GridView Control — CRUD
- Filtering, Sorting & DropDownList
- Hyperlinks & Row Deletion

Creating a Consistent Website

Why Consistency Matters in Web Design

The Problem

- Each page styled manually
- CSS duplication everywhere
- Changing one color = editing 100 files
- Inconsistent look across pages
- Navigation repeated on every .aspx
- Maintenance nightmare

The Solution

- ASP.NET Themes — centralized styles
- Skins — control-specific appearance
- Master Pages — shared layout
- ContentPlaceHolder — page-specific content
- Single edit → all pages update
- Professional, uniform look

Benefits

- DRY principle (Don't Repeat Yourself)
- Faster development
- Easy redesign
- Team consistency
- Better UX
- Lower maintenance cost

ASP.NET 2.0 — Themes

Themes provide a centralized way to define and apply consistent look-and-feel across an entire ASP.NET website.

What is a Theme?

- A folder inside App_Themes with .css and .skin files
- Applied globally via web.config or per-page via @Page directive
- Can define styles for every ASP.NET server control

Theme Folder Structure

```
App_Themes/
  ├── BlueTheme/
  │   ├── StyleSheet.css
  │   ├── Controls.skin
  └── RedTheme/
      ├── StyleSheet.css
      └── Controls.skin
```

Applying a Theme:

```
<%@ Page Theme="BlueTheme" ... %>

<!-- or in web.config -->
<pages theme="BlueTheme" />
```

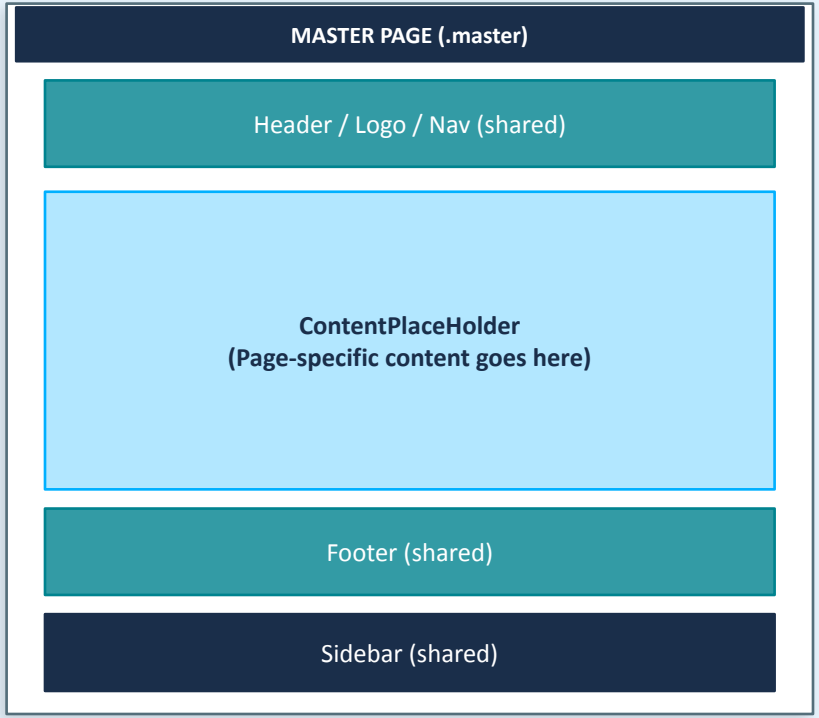
Sample .skin File:

```
<asp:Button runat="server"
   BackColor="#003399"
   ForeColor="White"
   Font-Name="Arial" />

<asp:Label runat="server"
   ForeColor="#003399"
   Font-Bold="True" />
```

ASP.NET 2.0 — Master Pages

Master Pages define the shared layout structure of a website — header, nav, footer and common UI elements.



Master Page syntax (.master):

```
<%@ Master Language="C#" %>
<html><body>
  <div>Common Header</div>
  <asp:ContentPlaceHolder
    ID="MainContent" runat="server"/>
  <div>Footer</div>
</body></html>
```

Content Page (.aspx):

```
<%@ Page MasterPageFile=
  "~/Site.master" %>

<asp:Content
  ContentPlaceHolderID="MainContent"
  runat="server">
   <!-- Your page content -->
</asp:Content>
```

Introducing the GridView Control

The GridView is the most powerful data-display control in ASP.NET — it renders tabular data with built-in sorting, paging, editing and deleting.

Key Features

- Automatic column generation
- Built-in sorting & paging
- Edit, Update, Delete rows
- Custom column templates
- Row selection support
- Data-binding to any source

Data Sources

- SqlDataSource
- ObjectDataSource
- XmlDataSource
- LinqDataSource
- DataTable / DataSet
- List<T> collections

Column Types

- BoundField — plain text
- ButtonField — button in cell
- CheckBoxField — boolean
- HyperLinkField — clickable link
- ImageField — image
- TemplateField — custom HTML

Basic GridView Declaration:

```
<asp:GridView ID="gvStudents"
   runat="server"
   DataSourceID="SqlDS1"
   AutoGenerateColumns="True"
   AllowSorting="True"
   AllowPaging="True" PageSize="10" />
```

GridView Events:

```
RowDataBound    // after row bound
RowDeleting     // before delete
RowDeleted      // after delete
RowEditing      // before edit
RowUpdating     // before update
SelectedIndexChanged
```

Filter Data in the GridView Control

Filtering allows users to view only the records they need — improving usability and performance.

Filtering Approaches

- WHERE clause in SqlDataSource
- FilterExpression property
- ControlParameter binding
- QueryString parameters
- Code-behind dynamic query
- LINQ .Where() clause

Method 1 — SqlDataSource FilterExpression:

```
<asp:SqlDataSource ID="SqlDS1" runat="server"
   ConnectionString="<%$ Conn %>"
   SelectCommand="SELECT * FROM Students"
   FilterExpression="City='{0}'">
   <FilterParameters>
      <asp:ControlParameter ControlID="txtCity"
         PropertyName="Text" />
   </FilterParameters>
</asp:SqlDataSource>
```

Method 2 — Code-Behind Dynamic WHERE:

```
protected void btnFilter_Click(...) {
  string city = txtCity.Text;
  SqlDS1.SelectCommand =
    "SELECT * FROM Students WHERE City='" + city + "'";
  gvStudents.DataBind();
}
```

Best Practice — Parameterized Query:

```
// ALWAYS use parameters — prevents SQL Injection!
cmd.CommandText =
  "SELECT * FROM Students WHERE City=@city";
cmd.Parameters.AddWithValue("@city", txtCity.Text);
```

Allow Users to Select from a DropDownList in the Grid

Combine a DropDownList with GridView to give users intuitive, pre-defined filter options from a list of values.

Approach 1 — Static DropDownList + Filter:

```
<asp:DropDownList ID="ddlDept"
   runat="server" AutoPostBack="True"
   OnSelectedIndexChanged="ddl_Changed">
   <asp:ListItem Value="All">All</asp:ListItem>
   <asp:ListItem Value="CS">CS</asp:ListItem>
   <asp:ListItem Value="IT">IT</asp:ListItem>
</asp:DropDownList>
```

Approach 2 — Dynamic DropDownList from DB:

```
<asp:SqlDataSource ID="ddlSource"
   SelectCommand=
    "SELECT DISTINCT Dept FROM Students"/>

<asp:DropDownList DataSourceID="ddlSource"
   DataTextField="Dept"
   DataValueField="Dept" />
```

Code-Behind — Handle Selection Change:

```
protected void ddlDept_Changed(object s, EventArgs e) {
   string dept = ddlDept.SelectedValue;
   if (dept == "All")
      SqlDS1.FilterExpression = "";
   else
      SqlDS1.FilterExpression = "Dept='" + dept + "'";
   gvStudents.DataBind();
}
```

Important Properties

- AutoPostBack="True" — triggers postback on change
- AppendDataBoundItems — adds static items to dynamic list
- SelectedValue — get currently selected value
- DataTextField — column to display in list
- DataValueField — column to use as value

Add a Hyperlink to the GridView

HyperLinkField and TemplateField allow you to add clickable links to each row — enabling master-detail navigation.

Method 1 — HyperLinkField (Declarative):

```
<asp:GridView ID="gvStudents" ...>
 <Columns>
  <asp:HyperLinkField
    HeaderText="Details"
    Text="View Details"
    DataNavigateUrlFields="StudentID"
    DataNavigateUrlFormatString=
      "Details.aspx?id={0}" />
 </Columns>
</asp:GridView>
```

Receiving the value on Details.aspx:

```
// On Details.aspx code-behind:
protected void Page_Load(object sender, EventArgs e) {
   int id = Convert.ToInt32(Request.QueryString["id"]);
   // Load student details using this id
   LoadStudentDetails(id);
}
```

Method 2 — TemplateField (Full Control):

```
<asp:TemplateField HeaderText="Action">
 <ItemTemplate>
  <asp:HyperLink runat="server"
    Text='<%# Eval("Name") %>'
    NavigateUrl='<%# String.Format(
      "Details.aspx?id={0}",
      Eval("StudentID")) %>'/>
 </ItemTemplate>
</asp:TemplateField>
```

Key Attributes

- DataNavigateUrlFields — DB column for URL param
- DataNavigateUrlFormatString — URL template with {0}
- Text / DataTextField — link display text
- Target — "_blank" to open in new tab
- Eval("FieldName") — data binding expression

Deleting a Row in GridView

GridView provides built-in delete support — with a simple configuration you can add delete functionality without writing complex code.

Step 1 — Configure SqlDataSource with DELETE:

```
<asp:SqlDataSource ID="SqlDS1" runat="server"
   SelectCommand="SELECT * FROM Students"
   DeleteCommand=
     "DELETE FROM Students WHERE StudentID=@StudentID">
   <DeleteParameters>
     <asp:Parameter Name="StudentID" />
   </DeleteParameters>
</asp:SqlDataSource>
```

Step 2 — Enable Deleting in GridView:

```
<asp:GridView ID="gvStudents"
   DataSourceID="SqlDS1"
   DataKeyNames="StudentID"
   AutoGenerateDeleteButton="True" />

<!-- or add a CommandField column: -->
<asp:CommandField ShowDeleteButton="True"
   HeaderText="Action" />
```

Step 3 — Code-Behind with Confirmation:

```
protected void gv_RowDeleting(object s,
GridViewDeleteEventArgs e) {
   // Get key value
   int id = (int)gvStudents.DataKeys[e.RowIndex].Value;
   // Can perform validation here
   if (!CanDelete(id)) {
      e.Cancel = true; // Stop the delete
      lblMsg.Text = "Cannot delete this record.";
   }
}
```

Important Notes

- DataKeyNames MUST be set to the primary key
- Set e.Cancel = true to abort the delete
- Add JS confirm() for user-friendly deletion
- Always refresh GridView after deletion
- Consider soft-delete (IsDeleted flag) over hard delete
- Log deletions for audit trails

Handling Errors in GridView Operations

Proper error handling prevents crashes, provides user feedback, and maintains data integrity during GridView CRUD operations.

Handling Errors in RowDeleted Event:

```
protected void gv_RowDeleted(object s,
                  GridViewDeletedEventArgs e) {
  if (e.Exception != null) {
    // Handle the error gracefully
    lblError.Text = "Delete failed: " + e.Exception.Message;
    lblError.Visible = true;
    e.ExceptionHandled = true; // Prevent crash
  } else {
    lblMsg.Text = "Record deleted successfully!";
    gvStudents.DataBind(); // Refresh grid
  }
```

Try-Catch in Code-Behind:

```
protected void DeleteRecord(int id) {
  try {
    string conn = ConfigurationManager
      .ConnectionStrings["DB"].ToString();
    using (SqlConnection c = new
SqlConnection(conn)) {
      SqlCommand cmd = new SqlCommand(
        "DELETE FROM Students WHERE ID=@id", c);
      cmd.Parameters.AddWithValue("@id", id);
      c.Open(); cmd.ExecuteNonQuery();
    }
```

Common Exceptions

- SqlException — DB constraint violation
- InvalidCastException — wrong data type
- NullReferenceException — null control
- FormatException — invalid conversion

Best Practices

- Always check e.Exception != null
- Set e.ExceptionHandled = true
- Show user-friendly error messages
- Log exceptions (EventLog / NLog)

Page-Level Error

- Use Page_Error event for unhandled errors
- Configure customErrors in web.config
- Redirect to friendly error page
- Never expose stack traces to users

Putting It All Together — Full Application Flow

Architecture of a complete ASP.NET application using Themes, Master Pages and GridView

Labels
Theme Applied

Layout Shell
Wraps all pages

GridView + DDL
Filter & Delete

SqlDataSource
CRUD operations

Implementation Checklist

- ✅ **Theme** — Create App_Themes folder → add CSS + .skin → set in web.config
- ✅ **GridView** — Add GridView → connect SqlDataSource → set DataKeyNames → enable paging/sorting
- ✅ **Hyperlink** — Use HyperLinkField or TemplateField with DataNavigateUrlFormatString
- ✅ **Master Page** — Create Site.master → add ContentPlaceHolder → link all .aspx via MasterPageFile
- ✅ **Filter** — Add DropDownList with AutoPostBack → use ControlParameter or FilterExpression
- ✅ **Delete** — Set DeleteCommand in SqlDataSource → handle RowDeleted event → show feedback

Summary & Key Takeaways

🎨 Themes & Skins

- App_Themes folder ⟶ CSS + .skin
- Applied via @Page or web.config
- Skins style server controls
- Multiple themes per project

📄 Master Pages

- Site-wide layout template
- .master file extension
- ContentPlaceHolder + Content tags
- MasterPageFile in @Page directive

📊 GridView Basics

- Binds to any data source
- Auto paging & sorting
- DataKeyNames for primary key
- AutoGenerateColumns property

🔍 Filtering & DDL

- FilterExpression + ControlParameter
- DropDownList AutoPostBack=True
- SelectedIndexChanged event
- Parameterized queries (security!)

🔗 HyperLinks

- HyperLinkField — declarative
- TemplateField — flexible
- DataNavigateUrlFormatString
- Eval() for data binding

🗑️ Delete & Errors

- DeleteCommand in SqlDataSource
- RowDeleting / RowDeleted events
- e.Cancel to abort
- e.ExceptionHandled to catch errors

Quick Reference — Properties & Events Cheat Sheet

Theme & Master Page Properties

Property/Directive	Purpose
<%@ Master %>	Declares a .master file
MasterPageFile="~/x.master"	Links a content page to master
ContentPlaceHolderID	Matches master and content
<%@ Page Theme="T" %>	Applies theme to single page
<pages theme="T" />	Applies theme to all pages (web.config)
SkinID="Primary"	Named skin for specific controls

GridView Key Properties

Property	Description
DataKeyNames	Primary key column(s)
AllowPaging / PageSize	Enable paging, set page size
AllowSorting	Enable column header sorting
AutoGenerateColumns	Auto-create columns from data
AutoGenerateDeleteButton	Add delete links automatically
EmptyDataText	Text shown when no records

GridView Events Quick Reference

Event	When it Fires	Key EventArgs Property
RowDataBound	After each row is data-bound	e.Row (GridViewRow)
RowDeleting	Before delete operation	e.RowIndex, e.Cancel
RowDeleted	After delete operation	e.Exception, e.ExceptionHandled
RowEditing	Before entering edit mode	e.NewEditIndex
RowUpdating	Before update operation	e.NewValues, e.OldValues
PageIndexChanging	Before page changes	e.NewPageIndex